

Case Study No.: 5

1. Case Study Title: To study the functioning of arrays and loops in JavaScript.

Create and manipulate arrays: push, pop, shift, unshift, splice, slice.

Use array methods: map(), filter(), reduce(), and forEach().

Create an array of numbers and find the maximum and minimum values.

```
// 1. Create an Array
let fruits = ["apple", "banana", "cherry"];

// push() - Adds an element to the END
fruits.push("mango");
// Result: ["apple", "banana", "cherry", "mango"]

// pop() - Removes the LAST element
let lastFruit = fruits.pop();
// Result: ["apple", "banana", "cherry"] | Removed: "mango"

// unshift() - Adds an element to the BEGINNING
fruits.unshift("strawberry");
// Result: ["strawberry", "apple", "banana", "cherry"]

// shift() - Removes the FIRST element
```

```
let firstFruit = fruits.shift();
// Result: ["apple", "banana", "cherry"] | Removed: "strawberry"

// splice(startIndex, deleteCount, ...itemsToAdd) - Modifies original
// array (Mutates)
// Example: Remove 1 element at index 1 and insert "grape"
fruits.splice(1, 1, "grape");
// Result: ["apple", "grape", "cherry"]

// slice(startIndex, endIndex) - Extracts a portion WITHOUT modifying
// original (Non-mutating)
// Note: endIndex is non-inclusive
let subArray = fruits.slice(0, 2);
// subArray: ["apple", "grape"]
// fruits stays: ["apple", "grape", "cherry"]

const numbers = [1, 2, 3, 4, 5, 6];

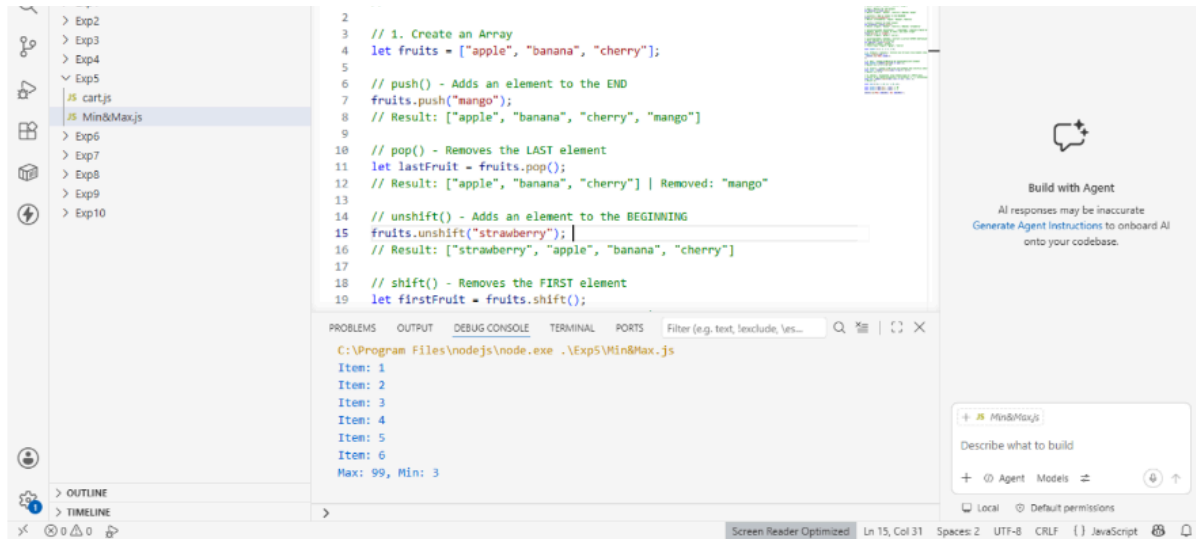
// 1. forEach() - Executes a function once for each array element (returns
// nothing)
numbers.forEach((num) => {
  console.log(`Item: ${num}`);
});

// 2. map() - Creates a NEW array by transforming every element
const doubled = numbers.map((num) => num * 2);
// Result: [2, 4, 6, 8, 10, 12]

// 3. filter() - Creates a NEW array with elements that satisfy a
// condition
const evens = numbers.filter((num) => num % 2 === 0);
// Result: [2, 4, 6]

// 4. reduce() - Accumulates array elements down to a SINGLE value
```

3. Output:



4. Result/Conclusion: This experiment demonstrated the core mechanics of JavaScript arrays and iterative processing.

Highlighting how different methods handle data manipulation, transformation, and evaluation. Modifying arrays using `push()`, `pop()`, `shift()`, `unshift()`, `splice()`, and `slice()` established a practical understanding of element positioning and the distinction between mutating original data versus returning new copies. Furthermore, utilizing functional array methods like `forEach()`, `map()`, `filter()`, and `reduce()` illustrated a clean, declarative alternative to traditional looping for transforming and aggregating data. Finally, calculating the maximum and minimum values of a number array using `Math.max()/Math.min()` with the spread operator, `reduce()`, and classic `for` loops demonstrated how algorithmic logic and modern language features can solve fundamental computational problems efficiently.